Taller evaluable de la sesión 4

Sistema de nómina del INTEP

Campo	Detalle
Resultado de aprendizaje	RA1 · Explico los principios fundamentales de la POO y su aplicación en el diseño de software
Desempeño evaluado	Comprendo y explico los conceptos clave de la POO: herencia
Conceptos aplicados	Relación «es un», extends, super, protected, sobrescritura con @Override, jerarquía en UML
Modalidad	Individual. Trabajo autónomo
Tiempo estimado	90 minutos
Plazo de entrega	Una semana. Entrega por GitHub
Valor	100 puntos

NOMBRE: GUILLERMO ALEJANDRO VARGAS DOCUMENTO: 1113788653

URL DE SU REPOSITORIO: https://github.com/Yussopp/Nomina.git

Antes de empezar

Este taller asume que usted ya recorrió la presentación de la sesión 4 y amplió la clínica veterinaria con las clases Perro y Gato. Si no lo ha hecho, hágalo primero: aquí no se repiten las explicaciones, se aplican.

Trabaje en el orden de las partes. La Parte B se resuelve en papel o en draw.io antes de programar: decidir la jerarquía **antes** de escribir código es el hábito profesional que este taller quiere formar.

Contexto del problema

ENUNCIADO

«El INTEP necesita un sistema para calcular su nómina. De todo empleado se registra su cédula, su nombre y la dependencia a la que pertenece, y de todos se puede obtener un resumen con esos datos. Sin embargo, el pago mensual no se calcula igual para todos.

Un docente de hora cátedra tiene un número de horas asignadas y un valor por hora: su pago mensual es el producto de los dos. Un empleado administrativo tiene un salario básico fijo que se paga completo cada mes, y además cuenta con 15 días de vacaciones al año, de los cuales puede ir tomando días. No se le pueden descontar más días de los que le quedan disponibles.»

OBSERVE ANTES DE MODELAR

Hay datos y comportamientos que **todos** los empleados comparten, y otros que son exclusivos de cada tipo. Identificar esa división es exactamente lo que evalúa la Parte B. Léalo de nuevo subrayando con un color lo común y con otro lo específico.

Parte A. Comprensión de conceptos

20 puntos · Responda con sus palabras y cite su propio código.

1. Aplique la prueba del «es un» a su jerarquía: escriba las dos frases que la justifican. Luego escriba una frase con las mismas clases que no tenga sentido, y explique por qué esa relación sería incorrecta. (4 pts)

- Un Docente <<es un>> Empleado (porque hereda los atributos y comportamientos comunes como: cédula, nombre y dependencia).

- Un Administrativo <<es un>> Empleado (porque comparte los atributos).

- Un Administrativo <<es un>> Docente o viceversa. Aunque son roles particularmente parecidos, son independientes y cumplen funciones totalmente diferentes dentro de la institución

2. Explique qué hace la línea s u p e r (c e d u l a , n o m b r e , d e p e n d e n c i a) ; en su código y por qué debe ir en la primera línea del constructor. (4 pts)

- Qué hace: Invoca al constructor de la clase padre (Empleado) desde las clases hijas (Docente o Administrativo), pasándole los atributos heredados (cedula, nombre y dependencia).

- Por qué debe ir en la primera línea: Java exige por regla estricta que el objeto de la clase padre esté completamente construido antes de inicializar o manipular los atributos propios de la clase hija. Si se intentara colocar después, el compilador generará un error.

- Algo mas que se debe tener en cuenta es que al momento de usar <super> en las clases hijas y construir un objeto los atributos de <super> deben ir en el mismo orden en que fueron definidos los métodos y atributos. Es decir, si tenemos public Empleado (String nombre, String cedula), la clase hija debe public Docente (String nombre, String cedula){ super (nombre, cedula)}. En caso de quedar en orden distinto, pueden ocurrir 1 de 2 cosas: si tienen el mismo tipo de dato (String) aunque compile, los datos se guardan en diferentes campos dañando la información, por otro lado, si tienen diferente tipo de dato (String e Int) java arrojaría error de compilación.

3. Los atributos de la clase padre quedaron en p r o t e c t e d y no en p r i v a t e . Explique qué habría pasado si los dejaba en private, y cuál es la alternativa que evitaría usar protected. (4 pts)

- Private: Si los atributos del padre se hubieran declarado como private, las clases hijas (Docente y Administrativo) no habrían tenido acceso directo a ellos dentro de sus propios métodos.

- Alternativa para evitar usar protected: Mantener los atributos como private y utilizar métodos de acceso en la clase padre, tales como getCedula(), getNombre() y getDependencia() (encapsulamiento estricto).

4. Explique para qué sirve la anotación @ O v e r r i d e y qué error concreto le evita a usted como programador. (3 pts)

- Para qué sirve: Indica al compilador que el método debajo está sobrescribiendo (redefiniendo) un método heredado de su clase padre.
- Error concreto que evita: Le evita cometer errores tipográficos en el nombre del método o discrepancias en los tipos y cantidad de parámetros (por ejemplo, escribir mal calcularPagoMensual o cambiar un parámetro por error). Sin @Override, el compilador crearía un método totalmente nuevo en lugar de sobrescribir el del padre, lo que rompería el polimorfismo de forma silenciosa.

5. En su método r e s u m e n ( ) de las clases hijas usted llamó a s u p e r . r e s u m e n ( ) . Explique qué gana el programa frente a escribir otra vez el texto del padre. (3 pts)

- Qué se gana: Se aplica el principio de **reutilización de código** y un diseño mantenible. Al llamar a super.resumen(), se aprovecha la cadena de texto que ya genera la clase padre (Empleado). Si en el futuro se modifica la estructura de los datos del padre, el cambio se reflejará automáticamente en todas las clases hijas sin necesidad de duplicar texto ni reescribir código repetido.

6. ¿Qué diferencia hay entre sobrescribir (override) y sobrecargar (overload)? Dé un ejemplo de cada uno tomado de su propio código o del curso. (2 pts)

Diferencia principal:

- Sobrescribir (override): Ocurre entre clases relacionadas por herencia (padre e hija), donde la clase hija redefine un método heredado manteniendo exactamente la misma firma (nombre y tipos de parámetros).
- Sobrecargar (overload): Ocurre dentro de una misma clase, donde se definen varios métodos con el mismo nombre pero con diferentes parámetros (ya sea en cantidad o en tipo de datos).

Ejemplos:

- Ejemplo de Sobrescribir (override): El método calcularPagoMensual() definido en la clase padre Empleado es sobrescrito tanto en la clase Docente como en la clase Administrativo para aplicar sus propias fórmulas de pago.
- Ejemplo de Sobrecargar (overload): Podría darse en una clase si tuviéramos un método registrarAsistencia() y otro registrarAsistencia(String observaciones), teniendo el mismo nombre pero distinta firma.

Parte B. Diseño de la jerarquía

25 puntos · Se resuelve antes de programar.

B1. Tabla de reparto (10 pts)

Clasifique cada dato y cada comportamiento del enunciado. Esta tabla decide toda su implementación: si está bien hecha, el código casi se escribe solo.

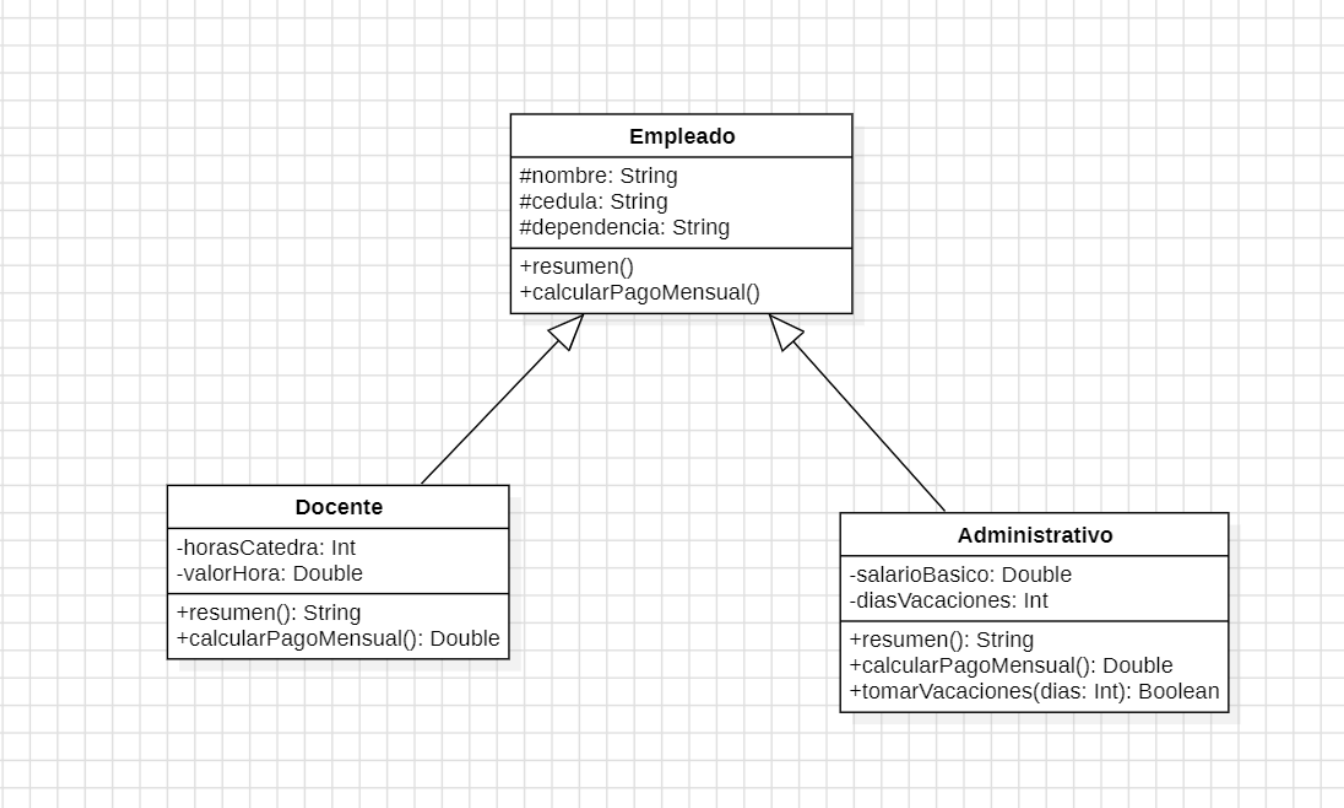
Dato o comportamiento	¿Dónde va?	Justificación
Nombre	Padre	Todo empleado posee un nombre de identificación.
Cedula	Padre	Atributo común a todo tipo de empleado del UNINTEP.
Dependencia	Padre	Todos los empleados pertenecen a una dependencia institucional.
Resumen()	Padre y sobrescrito	Todos los empleados muestran 3 datos base, las hijas agregan los suyos particulares.
calcularPagoMensual()	Padre y sobrescrito	Todos los empleados reciben un pago, pero la fórmula cambia según su rol.
horasCatedra	Hija(Docente)	Los docentes son los que trabajan por horas catedra
valorHora	Hija(Docente)	Al trabajar por horas catedra y no poseer salario fijo, se paga por horas.
salarioBasico	Hija(Administrativo)	Trabajan por un salario fijo
diasVacaciones	Hija(Administrativo)	Se les asignan días de vacaciones
tomarVacaciones	Hija(Administrativo)	Pueden hacer uso de las vacaciones dependiendo de los días disponibles que tenga cada uno.

En la columna del medio escriba **Empleado** (clase padre), **Docente**, **Administrativo**, o **Padre y sobrescrito** si es un método que existe en el padre pero cada hija redefine.

B2. Diagrama de clases UML (15 pts)

Dibuje la jerarquía completa. Debe contener:

- Las **tres clases** con sus tres compartimentos.
- La **flecha de herencia** de punta triangular hueca, que va de cada hija hacia el padre.
- Los atributos del padre con el símbolo **#** (protected) y los de las hijas con **–**.
- Los constructores de las tres clases, sin tipo de retorno.
- El método sobrescrito **repetido en las hijas**, que es como UML indica que fue redefinido.



Parte C. Implementación en Java

45 puntos · El código debe compilar y ejecutarse sin interrumpirse.

C1. Clase padre Empleado (10 pts)

- Tres atributos en `protected`: `cedula`, `nombre` y `dependencia`.
- Un constructor que reciba los tres y los asigne con `this`.
- Un método `resumen()` que devuelva un String con los tres datos.
- Un método `calcularPagoMensual()` que devuelva `double`. En el padre retorna 0: son las hijas las que saben calcularlo.

C2. Clase Docente (10 pts)

- Hereda de Empleado con `extends`.
- Agrega en `private`: `horasCatedra` y `valorHora`.
- Su constructor recibe los cinco datos y llama a `super(...)` en la primera línea.
- Sobrescribe `calcularPagoMensual()` con `@Override`: horas por valor de la hora.
- Sobrescribe `resumen()` usando `super.resumen()` y agregando sus datos.

C3. Clase Administrativo (15 pts)

- Hereda de Empleado. Agrega en `private`: `salarioBasico` y `diasVacaciones`.
- Su constructor recibe cédula, nombre, dependencia y salario básico. Los **15 días de vacaciones se asignan dentro del constructor**, no se reciben como parámetro.
- Método propio `tomarVacaciones(int dias)` que devuelva `boolean`: debe rechazar e informar por consola si los días son menores o iguales a cero, o si superan los disponibles.
- Sobrescribe `calcularPagoMensual()`: devuelve el salario básico.

- Sobrescribe `resumen()` usando `super.resumen()`.

C4. Clase de prueba Nomina (10 pts)

Escriba un main que demuestre, en este orden y con mensajes claros:

1. La creación de al menos un docente y un administrativo.
2. El resumen de ambos, mostrando que **heredaron** cédula, nombre y dependencia.
3. El pago mensual de cada uno, evidenciando que el **mismo método** calcula distinto en cada clase.
4. Una toma de vacaciones válida del administrativo.
5. Un intento de tomar más días de los disponibles, **rechazado con mensaje**.
6. El resumen final del administrativo, con los días que le quedan.

> **DATOS DE PRUEBA SUGERIDOS**
>
> Un docente con 60 horas a $35.000 debe dar un pago de **$2.100.000**. Un administrativo con salario básico de $1.800.000 debe dar exactamente eso. Si toma 5 días de vacaciones le deben quedar **10**. Úselos para comprobar su implementación.

RESTRICCIÓN IMPORTANTE

Ninguna clase hija puede volver a declarar `cedula`, `nombre` ni `dependencia`. Si lo hace, no está heredando: está duplicando, que es justamente el problema que la herencia resuelve. Ese error anula el criterio C2 o C3 según donde ocurra.

Parte D. Entrega en GitHub

10 puntos · El syllabus establece GitHub como medio del trabajo autónomo.

Requisito	Especificación
Nombre del repositorio	programacion2-taller-sesion4
Visibilidad	Público
Archivos de código	Empleado.java, Docente.java, Administrativo.java, Nomina.java
Diagrama	diagrama.png en la raíz del repositorio
Evidencia	salida.png con la ejecución completa, incluida la línea de rechazo
README.md	Respuestas de la Parte A y la tabla de reparto de la Parte B1

Rúbrica de evaluación

Léala antes de empezar: le dice con precisión qué se espera de usted.

Criterio	Excelente	Aceptable	Insuficiente	Pts
Comprensión (A)	Explica con lenguaje propio y cita su código	Explica correctamente sin citar su código	Repite definiciones o responde incompleto	20
Diseño de la jerarquía (B)	Reparto correcto y diagrama coherente con él	Reparto correcto con una o dos omisiones en el diagrama	Sube al padre algo que solo usa una hija, o falta la flecha de herencia	25
Clase padre (C1)	Atributos protected, constructor con this, los dos métodos	Correcto con una omisión menor	Atributos public, o falta calcularPagoMensual	10
Herencia y super (C2, C3)	extends, super en la primera línea, sin duplicar atributos del padre	Funciona pero el constructor no reutiliza super correctamente	Las hijas vuelven a declarar los atributos del padre	15
Sobrescritura (C2, C3)	@Override en los dos métodos y super.resumen() reutilizado	Sobrescribe sin @Override o repite el texto del padre	No sobrescribe: el pago se calcula igual para ambos	10
Validación y prueba (C3, C4)	tomarVacaciones rechaza los dos casos; el main demuestra los seis puntos	Demuestra el funcionamiento sin todos los rechazos	No valida, o el programa se interrumpe	10
Entrega (D)	Repositorio completo con todas las evidencias	Falta una evidencia	Faltan archivos o el repositorio es privado	10

CRITERIOS QUE AFECTAN TODO EL TALLER

- Si el proyecto **no compila**, la Parte C pierde el 50% del puntaje.
- Si las clases hijas duplican los atributos del padre, el criterio de herencia se califica como insuficiente aunque el programa funcione.
- Los nombres deben seguir la convención de Java: clases en mayúscula inicial, atributos y métodos en camelCase.
- Cada día de retraso descuenta 10 puntos.

Autoevaluación antes de entregar

Complete esta lista antes de subir su trabajo. El syllabus contempla autoevaluación como parte del proceso, y además le sirve para detectar lo que falta mientras todavía puede corregirlo.

Verifique	Sí	No
Mis dos clases hijas usan extends		
Ninguna clase hija vuelve a declarar cedula, nombre ni dependencia		
Los dos constructores de las hijas llaman a super en la primera línea		
Los atributos del padre están en protected y los de las hijas en private		
calcularPagoMensual() lleva @Override en las dos hijas		
resumen() de las hijas llama a super.resumen() en vez de repetir el texto		
tomarVacaciones rechaza días negativos y días que superan los disponibles		
Con 60 horas a $35.000 obtengo $2.100.000		
Mi diagrama tiene la flecha de punta triangular hueca hacia el padre		
Puedo explicar por qué cada atributo quedó donde quedó		

UNA PREGUNTA PARA DEJARLE PENSANDO

En su main usted tuvo que llamar a `d1.calcularPagoMensual()` y a `a1.calcularPagoMensual()` por separado, porque son variables de tipos distintos. Si el INTEP tuviera 200 empleados, ¿tendría que escribir 200 líneas? Esa pregunta es el tema completo de la sesión 5.